

Università
degli Studi
di Padova



UNIVERSITÀ DEGLI STUDI DI PADOVA

Dipartimento di Matematica "Tullio Levi-Civita"

Corso di Laurea in Informatica

Insegnamento di Ingegneria del Software

Anno Accademico 2026/2027

Valutazione Capitolati

Sintesi comparativa e motivazione della scelta

Gruppo	Bit Crusher
Email	bitcrusherteam@gmail.com
Versione	1.0.0
Data Redazione	28 Ottobre 2026
Responsabile Redazione	Alex Singh
Responsabile di Progetto	Riccardo Pozzato
Uso	Esterno

Registro delle Modifiche

Vers.	Data	Autore	Verificatore	Descrizione
1.0.0	28/10/2026	Alex Singh	Riccardo Pozzato / Antonio Tufano	Approvazione per il rilascio finale del documento.
0.1.5	28/10/2026	Alex Singh	Riccardo Pozzato	Completamento della sottosezione 4.1.4.
0.1.4	28/10/2026	Alex Singh	Riccardo Pozzato / Antonio Tufano	Aggiunta della sezione per la scelta del capitolato.
0.1.3	28/10/2026	Alex Singh	Riccardo Pozzato / Antonio Tufano	Espansione della sezione delle conclusioni e scelta finale.
0.1.2	28/10/2026	Alex Singh	Riccardo Pozzato	Fusione del documento di studio della fattibilità con la valutazione dei capitolati.
0.1.1	27/10/2026	Riccardo Pozzato Antonio Tufano Pietro L. Acampora	Alex Singh	Prima stesura e inserimento analisi dei capitolati.
0.1.0	26/10/2026	Riccardo Pozzato Antonio Tufano	Alex Singh	Impostazione dell'ossatura iniziale del documento.

Indice

Registro delle Modifiche **1**

1 Introduzione **3**
1.1 Scopo del Documento . . . 3
1.2 Criteri di Valutazione . . . 3

2 Sintesi delle Valutazioni **3**

3 Analisi Dettagliata per Capitolato **4**
3.1 C0: AIRA: AI Requests Automation . . . 4
3.2 C1: AURA: AI for Understanding, Reasoning and Automation . . . 4
3.2.1 Descrizione generale . . . 4
3.2.2 Obiettivi e finalità . . . 4
3.2.3 Tecnologie richieste . . . 4
3.2.4 Vincoli e requisiti . . . 5
3.2.5 Aspetti positivi . . . 5
3.2.6 Criticità e rischi . . . 5
3.3 C2: NEXUM Concierge . . . 5

3.4 C3: Fossdot: Smart Speaker Open Source estensibile e rispettoso della privacy . 6
3.4.1 Descrizione generale . . . 6
3.4.2 Obiettivi e finalità . . . 6
3.4.3 Tecnologie richieste . . . 6
3.4.4 Vincoli e requisiti . . . 6
3.4.5 Aspetti positivi . . . 7
3.4.6 Criticità e rischi . . . 7
3.5 C4: Gestione Veicoli e Servizi Post-Vendita . . . 7
3.6 C5: RareCare: Piattaforma AI per la raccolta dati e il supporto decisionale nelle malattie rare . . . 8
3.7 C6: Sistema AI di caching semantico e dashboard FinOps (Var Group) . . . 8
3.7.1 Descrizione generale . . . 8
3.7.2 Obiettivi e finalità . . . 8
3.7.3 Tecnologie richieste . . . 9
3.7.4 Vincoli e requisiti . . . 9
3.7.5 Aspetti positivi . . . 9
3.7.6 Criticità e rischi . . . 9
3.8 C7: SmartDashboard . . . 10
3.9 C8: Sonar: Documentazione vivente di uno store Shopify . . . 10
3.10 C9: SpecCheck: Skills OpenCode per tracciabilità tra casi d'uso, requisiti e codice sorgente . . . 11

4 Capitolato Scelto: Caching Semantico e FinOps (C6) 11
4.1 Motivazione Scelta . . . 11
4.1.1 Allineamento con le competenze del gruppo . . . 11
4.1.2 Interesse Tecnico e Professionale . . . 12
4.1.3 Rapporto Complessità/Fattibilità . . . 12
4.1.4 Qualità del rapporto con la proponente . . . 12
4.2 Strategie di mitigazione dei rischi . . . 12
4.3 Piano di formazione preliminare . . . 13
4.4 Impegno orario stimato . . . 14

5 Conclusioni e Scelta Finale 14

1 Introduzione

1.1 Scopo del Documento

Il presente documento espone il processo di valutazione effettuato dal gruppo **Bit Crusher** in merito ai capitolati d'appalto proposti per il Progetto Didattico. L'analisi ha portato alla selezione del progetto più in linea con le aspirazioni formative, le competenze pregresse e gli obiettivi di crescita del team.

1.2 Criteri di Valutazione

Per garantire un approccio oggettivo e sistematico, ogni proposta è stata valutata assegnando un punteggio da 1 (minimo) a 5 (massimo) sui seguenti quattro assi:

- **V1 - Interesse nel Dominio:** Attrattiva verso l'ambito applicativo e le tematiche del progetto.
- **V2 - Interesse Tecnologico:** Stimolo offerto dallo stack tecnologico e dalle architetture (Cloud, AI, ecc.).
- **V3 - Fattibilità e Complessità:** Bilanciamento tra la sfida proposta, le competenze attuali e i vincoli temporali.
- **V4 - Chiarezza e Supporto:** Qualità della documentazione iniziale e disponibilità dell'azienda durante gli incontri.

2 Sintesi delle Valutazioni

La tabella seguente riassume i punteggi assegnati dal gruppo a valle degli incontri con i referenti aziendali e delle riunioni interne.

Capitolato	V1	V2	V3	V4	Totale
C0: AIRA: AI Requests Automation	3	3	3	3	**12**
C1: AURA: AI for Understanding...	4	3	4	3	**14**
C2: Concierge	2	3	3	4	**12**
C3: Fossdot: Smart Speaker Open Source...	5	4	3	4	**16**
C4: Gestione Veicoli e Servizi Post-Vendita	2	2	4	3	**11**
C5: RareCare: Piattaforma AI...	4	3	3	4	**14**
C6: Sistema AI di caching semantico e dashboard FinOps	5	5	4	5	**19**
C7: Smart Dashboard: Piattaforma di BI...	3	4	3	3	**13**
C8: Sonar: Documentazione vivente...	3	3	4	3	**13**
C9: SpecCheck: Skills OpenCode...	2	3	3	3	**11**

Legenda: V1 = Dominio, V2 = Tecnologie, V3 = Complessità, V4 = Supporto. Il punteggio massimo ottenibile è 20.

3 Analisi Dettagliata per Capitolato

3.1 C0: AIRA: AI Requests Automation

Descrizione: Il progetto mira a realizzare un tool multimodale basato su Intelligenza Artificiale che gestisce le richieste B2B arrivate in forma non strutturata (e-mail con allegati, note vocali, immagini, messaggi). Il sistema monitora in modo continuo i canali configurati, applica un filtro di sicurezza per l'isolamento dei dati e utilizza speech-to-text, OCR e un LLM per estrarre entità e intenzioni, convertendole in dati strutturati. In base all'entità rilevata, avvia automaticamente azioni configurabili, come chiamate API REST verso B2B, ERP, CRM e ticketing o l'invio di e-mail di notifica, eliminando i passaggi manuali. Un'interfaccia amministrativa web consente di gestire i canali, consultare i log e configurare la persistenza dei dati.
Tecnologie: Java, Spring Boot, Angular, TypeScript, LangChain4j, LLM, Whisper, Google Speech, Tesseract, EasyOCR, SQLite/MariaDB/PostgreSQL, API REST, JSON/YAML

- **Pro:** Casi d'uso B2B reali; utilità aziendale immediata; integrazione IA pratica.
- **Contro:** Stack tecnologico imposto; scarsa libertà decisionale sul backend; logiche aziendali preesistenti rigide.

Sintesi: Impone uno stack legacy e logiche aziendali chiuse, negando la libertà di progettare un'architettura backend pulita e moderna in autonomia.

3.2 C1: AURA: AI for Understanding, Reasoning and Automation

3.2.1 Descrizione generale. AURA, proposto da Vimar S.p.A., chiede di collegare i servizi commerciali di IA generativa (Claude, ChatGPT, Copilot, Gemini) agli impianti domotici Vimar View Wireless, per interrogarli e comandarli in linguaggio naturale. L'impianto è una rete Bluetooth Mesh di dispositivi da incasso collegata, tramite gateway, al cloud Vimar, che espone la KNX IoT 3rd Party API: un'interfaccia REST standard in cui ogni dispositivo è un insieme di *funzionalità* con i propri *datapoint*, collocate in una *location*. Le macro-componenti sono tre: un'infrastruttura cloud-ready, un MCP server che fa da adattatore verso KNX IoT (con il suo client) e l'agente IA esterno con cui l'utente conversa. Vimar fornisce a ogni gruppo un kit fisico (luci, tapparelle, termostato, sensori UWB), l'accesso alle API e, se serve, spazio su AWS.

3.2.2 Obiettivi e finalità. La finalità è un dimostratore IoT in concomitanza all'AI che mostri fattibilità e potenzialità delle API Vimar: invece dell'ennesimo assistente dedicato, ognuno usa il servizio IA che preferisce attraverso standard aperti, in linea con la strategia aziendale di valorizzare le integrazioni di terze parti. Non si vuole sostituire l'app esistente, ma aprire un canale complementare e una base per evoluzioni future (l'app mobile mostrata è fuori scope però). In concreto: le tre macro-componenti funzionanti sui 17 casi d'uso base e sugli 11 avanzati obbligatori (ambiguità, sinonimi, richieste implicite, conferme, errori), più architettura C4, un'analisi sullo sviluppo basata sulle specifiche con stima dei costi AWS, documentazione MCP come sito statico, manuale d'installazione, release pubblica, demo e report dei test.

3.2.3 Tecnologie richieste. Vimar lascia ampio margine, ma i paletti sono netti: Docker con docker-compose per un IaC replicabile con un solo comando e un `config.env` documentato, compatibilità di rilascio su AWS, MCP per server e client, KNX IoT 3rd Party API (v2.1.0) con OAuth2, notifiche push via *Subscription* (niente polling, salvo accordi), un servizio IA commerciale e Git pubblico con licenza open source, occhio alle licenze delle dipendenze. Tra i suggerimenti troviamo Python con FastMCP (o TypeScript/Java, stesso SDK per client e server;

quale FastMCP intendano va chiarito, nella v2 dell'SDK ufficiale non c'è più), AWS Lightsail o EC2, la CI con GitHub Runners e Copilot con spec-kit.

3.2.4 Vincoli e requisiti. Conviene dividerli in obbligatori, opzionali e nice-to-have. Gli obbligatori seguono le macro-componenti: MCP server con gli endpoint KNX IoT Authorization, Datapoint, Funzionalità, Information, Location, Subscription e Timeseries, usabile da client di terze parti, con lettura/scrittura sull'impianto, ricostruzione delle relazioni tra entità e OAuth2; client MCP che parla solo con il nostro server; modello IA esterno con login MyVimar e impianto selezionato mantenuto lungo la conversazione. Poi i test: unità e integrazione con coverage minima del 75%, end-to-end almeno all'80% sui requisiti. Opzionali il deploy effettivo su AWS (anche serverless + container), l'endpoint Device, il client in VSCode o via CLI/TUI e Terraform, CDK o Ansible accanto al compose. Infine i nice-to-have: multi-impianto, azioni condizionate («se ci sono luci accese, spegnile»), scenari composti, storico con relative operazioni, trigger ed eventi.

3.2.5 Aspetti positivi. Dal confronto col gruppo emerge che AURA è relativamente complesso, e l'abbiamo concordato come un aspetto positivo: richiede di lavorare con standard e protocolli veri (MCP, KNX IoT, OAuth2), utili alla nostra crescita, partendo da ciò che oggi sta dilagando per portare la smart home allo stadio successivo. Ci ha colpito poi che Vimar non si affidi ad un'unità centralizzata come il classico Home Assistant, ma esponga un endpoint verso servizi di terzi (Anthropic, OpenAI, Google, Microsoft), con una separazione netta delle responsabilità tra le parti (nessuno vieta di affiancarci un Home Assistant).

3.2.6 Criticità e rischi. MCP corre veloce, e la cosa più spaventosa sono proprio i breaking change: la revisione del 28 luglio 2026 ne ha portati di pesanti (spariscono sessioni e handshake, la registrazione dinamica dei client è deprecata) e, nonostante la nuova policy di deprecazione a dodici mesi, i servizi commerciali possono parlare versioni diverse del protocollo: la compatibilità è a carico nostro. Il passaggio allo stateless risolve il conflitto con il load balancer ma sposta il problema: l'impianto selezionato non vive più nella sessione e va gestito con handle espliciti. L'autenticazione resta la criticità più grossa: due layer di token (client → MCP, MCP → KNX IoT) tra refresh, revoca e scope, niente di out-of-the-box. Il problema nascosto, emerso alla seconda riunione, è la risoluzione delle ambiguità: non è un limite dell'MCP ma di prompt engineering (una mitigazione l'abbiamo pensata, ma non è banale). Poi il compose che si fa fragile al crescere dell'infrastruttura, AWS (opzionale, attenzione) come collo di bottiglia di networking, prompt injection e token exposure (patata bollente con un repo pubblico) e la dipendenza da Vimar, la cui eventuale v2.2 dell'API dovrà essere retro-compatibile in caso.

Sintesi: Candidatura scartata in quanto non sufficientemente stimolante rispetto ad altre scelte.

3.3 C2: NEXUM Concierge

Descrizione: Il progetto Concierge, proposto da Eggon, mira a realizzare un assistente virtuale conversazionale per la piattaforma di servizi HR NEXUM. Il tool consentirà ai dipendenti di accedere a informazioni e completare operazioni, come la richiesta di ferie e permessi, direttamente via chat. Il sistema sfrutta un modello linguistico per mantenere il contesto, consultare la knowledge base documentale (regolamenti e manuali) e interfacciarsi con i servizi aziendali per la lettura dei dati e l'invio delle richieste. Include inoltre un backoffice per gestire i documenti aziendali.
Tecnologie: Angular, TypeScript, PrimeNG, Tailwind CSS, Ruby on Rails, PostgreSQL, Redis, Sidekiq, LLM/stack AI (da concordare), OAuth2, JWT

- **Pro:** Mentorship e supporto costanti; architettura di base già definita; forte focus sulla User Experience.
- **Contro:** Interazione forzata con backend chiusi; dipendenza totale da API esterne; sviluppo sbilanciato sulle logiche frontend.

Sintesi: L'obbligo di appoggiarsi a un backend "black-box" aziendale preesistente azzera totalmente la possibilità di strutturare il database e le API da zero.

3.4 C3: Fossdot: Smart Speaker Open Source estensibile e rispettoso della privacy

3.4.1 Descrizione generale. FossDot, proposto da Uranion EMEA Ltd, chiede uno smart speaker open source, funzionalmente analogo ad un Amazon Echo Dot, coprendo l'intero stack dall'hardware ai servizi. Alla base c'è l'hardware di riferimento (SBC o microcontrollore, array di microfoni, altoparlante, amplificatore e feedback luminoso), da racchiudere in un contenitore da prodotto finale; sopra, il firmware acquisisce l'audio, rileva la wake word (o il push-to-talk), invia lo stream al server e riproduce la risposta. Dopodiché il servizio di bridging implementa la pipeline STT $\rightarrow$ LLM $\rightarrow$ TTS con un'astrazione che rende il provider IA (cloud o locale) sostituibile, più un client MCP per servizi esterni, Skills e Agents; infine un'interfaccia web registra i dispositivi, li associa a un account e ne gestisce provider e interconnessione.

3.4.2 Obiettivi e finalità. Ciò a cui mira è un'alternativa aperta agli smart speaker commerciali, chiusi e legati al cloud del produttore e ad un unico assistente: il provider IA si sceglie e si sostituisce liberamente, il sistema è self-hosted e ispezionabile, può elaborare la voce tutta in locale (utile anche dove il GDPR è stringente) ed è ricostruibile da chiunque, per cui non muore quando qualcuno spegne un cloud. L'MVP chiede hardware validato con distinta base e prototipo, firmware con wake word o push-to-talk, bridging verso almeno un provider, registrazione di un dispositivo e flusso end-to-end su dispositivo fisico. La consegna completa aggiunge provider commutabili (es. Ollama), MCP con uno o due servizi esterni, Skills e Agents a voce, multi-dispositivo (stanze, annunci, riproduzione sincronizzata, intercom), firmware in Rust e/o OTA e hardening (segreti, autenticazione, STT/TTS locali).

3.4.3 Tecnologie richieste. Lo stack non è imposto: tocca al gruppo scegliere e motivare rispetto al caso d'uso. Fatto sta che dei punti fermi ci sono: rilascio dell'intero progetto (software e sorgenti hardware) con licenza permissiva, Apache 2.0 o MIT, MCP lato bridging e un'applicazione web per la gestione; il firmware in Rust è preferenziale (un valore aggiunto, non un vincolo) e sono incoraggiate containerizzazione e automazione di build e rilascio. I riferimenti, non prescrittivi, sono Rust embedded con Embassy, Raspberry Pi, Seeed ReSpeaker, openWakeWord, Silero VAD, faster-whisper, Piper, Ollama e Axum/Tokio, con Willow e Rhasspy/Home Assistant come prior art.

3.4.4 Vincoli e requisiti. La licenza è un punto di pressione: permissiva, con la proprietà che resta al gruppo e un repository pubblico. La validazione va fatta su hardware reale (wake word $\rightarrow$ domanda $\rightarrow$ risposta su dispositivo fisico), la simulazione è ammessa solo nei test. La proponente non fornisce dataset e i flussi vocali sono dati personali, quindi il trattamento va progettato secondo il GDPR. I test vanno concordati per ogni componente, con coverage di riferimento all'80% e report per bridging e interfaccia, mentre per il firmware (situazione più delicata) si decide caso per caso; le scelte vanno documentate con motivazioni e problemi aperti. Restano impliciti, con priorità da concordare, il canale dispositivo-server e la scoperta in rete.

3.4.5 Aspetti positivi. Il perimetro è modulare: la scelta tra MVP e consegna completa spetta a noi, quindi preventivo e rischio si dimensionano sulle nostre capacità reali. La libertà tecnologica non è un salto nel vuoto, perché il capitolato indica librerie e prior art concreti, e l'architettura si scompone bene, sia per lavorare in parallelo in sette sia per pattern riconoscibili come Strategy e Adapter sui provider. I criteri di accettazione sono verificabili, non servono dati né accessi della proponente, la pipeline può girare tutta in locale e la fattibilità è già dimostrata da dispositivi open come Home Assistant Voice PE (59 €). Da non sottovalutare il supporto, tra formazione, SAL e tech dive con i loro software architect, e il valore di portfolio di un prodotto fisico tutto nostro.

3.4.6 Criticità e rischi. Il primo nodo è l'architettura del dispositivo: i riferimenti puntano a due strade incompatibili (microcontrollore con Rust `no_std` ed Embassy, oppure SBC Linux con openWakeWord in Python) e la scelta decide wake word, firmware e distinta base, quindi va chiusa con una PoC prima dell'RTB. L'audio a distanza (eco, beamforming, rumore) è elaborazione del segnale, fuori dal nostro percorso: senza un DSP integrato la wake word scatta a vuoto. Rust embedded è maturo solo in parte (in esp-hal 1.0 I2S e Wi-Fi hanno API ancora instabili) e i motori di wake word su MCU sono in C/C++. Ci sono poi ambiguità (l'MVP ammette il push-to-talk ma il collaudo è «wake word → risposta»; mancano latenza, lingua e soglie di falsi positivi), licenze che stridono con Apache/MIT (modelli di openWakeWord in 4.0, Piper passato a GPLv3), hardware e costi API non attribuiti e un supporto centrato su AI e software proprio dove il rischio è embedded. Infine la privacy, con un microfono sempre in ascolto.

Sintesi: Tenuto in considerazione ma superato da proposte maggiormente in linea con i nostri interessi backend.

3.5 C4: Gestione Veicoli e Servizi Post-Vendita

Descrizione: Il progetto, proposto da M31, mira a sviluppare una piattaforma distribuita per la gestione dei veicoli e dei servizi post-vendita nel settore automotive. Il sistema connette la casa automobilistica (cataloghi, veicoli e campagne di richiamo), le concessionarie (vendite e manutenzioni) e i proprietari (area personale, consultazione dati e appuntamenti).

Tecnologie: Node.js, Nest.js, TypeScript, Go, Angular, PostgreSQL, MongoDB, Redis, NATS o Apache Kafka, Kubernete, Prometheus e Grafana

- **Pro:** Architettura a microservizi moderna; modellazione dati relazionale estesa; settore automotive in forte crescita.
- **Contro:** Dominio dati estremamente frammentato; rischio di logiche di astrazione eccessive; integrazioni di rete complesse.

Sintesi: L'astrazione spinta dei microservizi frammenta inutilmente il sistema, quando una base di dati relazionale solida e centralizzata sarebbe nettamente più efficiente.

3.6 C5: RareCare: Piattaforma AI per la raccolta dati e il supporto decisionale nelle malattie rare

Descrizione: Il progetto si pone l'obiettivo di sviluppare RareCare Navigator, una piattaforma digitale intelligente pensata per supportare la gestione delle malattie rare e complesse (focalizzandosi inizialmente sulla Sindrome di Sanfilippo). Il sistema mira a superare l'isolamento e la frammentazione dei servizi vissuti dalle famiglie, agendo come connettore sicuro tra centri clinici, terapisti, scuole, servizi sociali e potenziando la figura del Care Navigator. La piattaforma integra un assistente virtuale basato su IA (RAG) vincolato a fonti verificate, checklist dinamiche, un fascicolo documentale cifrato e un modello modulare a plugin (Disease Pack) per replicare facilmente il sistema su altre patologie.

Tecnologie: Flutter o React Native, AWS Lambda, API Gateway, Amazon S3, EventBridge e Aurora Serverless v2 con estensione, Amazon Bedrock, Amazon Cognito e Amazon Verified Permissions con linguaggio Cedar.

- **Pro:** Forte impatto sociale; dominio medico molto affascinante.
- **Contro:** Complessità normativa per la gestione di dati sensibili sanitari.

Sintesi: I rigidi limiti burocratici e i vincoli legali sui dati sanitari (GDPR) rallentano l'ingegneria del software costringendo a continui compromessi.

3.7 C6: Sistema AI di caching semantico e dashboard FinOps (Var Group)

3.7.1 Descrizione generale. Il capitolato riguarda principalmente lo sviluppo di un sistema di caching semantico e di una dashboard FinOps in estensione a LiteLLM, il proxy open source per il routing multi-provider è già in produzione presso Var Group. L'obiettivo è in sostanza colmarne due limiti critici: la cache riconosce solo query identiche al 100% (quindi si ripagano chiamate per domande semanticamente equivalenti ma testualmente diverse) e manca un riscontro chiaro su costi e consumi, senza aggregazioni per team e progetti né un cruscotto per il consuntivo. Si chiede quindi un layer di caching basato su embedding e similarità (con soglia tarabile, invalidazione per TTL e versionamento, persistenza su Redis o vector DB come Qdrant) che intercetti le query semanticamente simili e ne restituisca la risposta in cache, e un servizio FinOps separato che aggreghi i consumi per team, progetto e utente quasi in tempo reale, calcoli il risparmio reale prodotto dalla cache, gestisca budget e alert e preveda la spesa mensile. L'MVP è un end-to-end funzionante (proxy, cache e dashboard integrati), accompagnato da architettura con ADR, documento dei requisiti post design thinking, diagrammi UML, test automatici con copertura almeno al 70% e repository Git con documentazione e bug report post-collaudo.

3.7.2 Obiettivi e finalità. L'obiettivo principe è ridurre i costi di inferenza verso gli LLM intercettando le query semanticamente sovrapponibili (il capitolato stima che circa l'80% delle query LLM sia semanticamente ridondante) e restituendo risposte dalla cache invece di ricontattare LLM di terzi. Oltre a ciò si chiede visibilità e controllo economico quasi in tempo reale sui consumi di token, superando lo «scopri il costo a fine fattura». Il tutto estendendo LiteLLM senza reinventare la ruota, con componenti originali, e dentro un contesto, quello Var Group, fatto di sovranità del dato e compliance (GDPR, NIS2, AI Act), con risorse condivise tra tenant ma dati isolati tra team. Le finalità invece sono di far evolvere LiteLLM da semplice proxy di routing ad orchestratore cosciente dei consumi, per ottimizzare il risparmio e rendere visibile dove finisce la spesa; dimostrare pattern architetturali estendibili (non solo con le soluzioni suggerite) e riusabili per funzionalità future del proxy; e, ultima ma non meno importante, la

formazione, dato che il progetto richiede molto design thinking, stand-up periodici, il panorama dei modelli open source e un contatto diretto con il mondo del Machine Learning.

3.7.3 Tecnologie richieste. Le tecnologie si dividono in domini interdipendenti, e va detto subito che il capitolato le indica come consigliate: l'unico vincolo vero è LiteLLM come proxy base. Per il proxy lo stack dichiarato è Python 3.10+, LiteLLM (sfruttando le funzioni personalizzate di lettura e scrittura della cache), FastAPI o Flask per gli endpoint e Docker con Docker Compose per ambienti riproducibili. Per gli embedding sono suggeriti text-embedding-3-small di OpenAI o sentence-transformers (open source), con similarità del coseno via numpy/scikit-learn; per il caching invece Redis (cache e TTL), Qdrant o pgvector come vector store e PostgreSQL per le metriche FinOps. Per la dashboard React con Chart.js oppure Streamlit (molto più diretto), su API REST JSON; infine il versionamento su Git, con GitHub o Bitbucket.

3.7.4 Vincoli e requisiti. Architetturalmente LiteLLM resta il proxy base senza reinventarlo, tutta la logica originale sostanziale va su cache semantica e FinOps e il design deve essere modulare ed estendibile; infrastrutturalmente si lavora in multi-tenancy, con dati isolati tra team e progetti. Sul processo sono previsti design thinking, User Story Mapping e una PoC di fattibilità prima dell'implementazione, più stand-up periodici con il referente. Qualitativamente la coverage deve essere almeno al 70% e, cosa fondamentale, i test deterministici vanno separati dalla valutazione probabilistica (soprattutto se più modelli di ML si corroborano a vicenda). Come requisiti funzionali, la cache deve riconoscere query semanticamente simili e non solo identiche, con una soglia tarabile sui dati (forniti e costruiti da noi) e quindi configurabile, invalidare le entry per TTL e versioning e loggare token e latenza risparmiati; la dashboard deve aggregare i consumi per team, progetto e utente, calcolare il risparmio reale prodotto dalla cache, gestire budget con alert automatici in caso di sforamento, prevedere la spesa mensile e aggiornarsi quasi in tempo reale. GDPR, NIS2, AI Act e sovranità del dato vengono dal contesto più che dai vincoli espliciti: se serva un deploy privato su AWS lo chiariremo al design thinking.

3.7.5 Aspetti positivi. Il problema posto è reale per il mercato AI enterprise (costi e GDPR) e stuzzica l'interesse di tutto il gruppo: disegnare una soluzione originale allo spreco di token è senza dubbio complesso, e la complessità è tale da richiedere competenze tecniche interdisciplinari, su un'architettura moderna e molto formativa. Oltretutto lo scope è ben delimitato (anche se con qualche vincolo ambiguo): la proponente non chiede di reinventare LiteLLM, ma di concentrare le energie sui due buchi che contano per loro, isolabili in sottoproblemi testabili ognuno con le proprie metriche. Da non sottovalutare il processo, tra design thinking, PoC, User Story Mapping, implementazione e collaudo, con formazioni dedicate (LiteLLM, embedding e caching, FinOps) e una Q&A ogni due settimane, che formano un canale di comunicazione e feedback strutturato; Var Group, del resto, si è dimostrata molto preparata, con supporto continuo e una pianificazione chiara degli incontri. Infine i criteri di accettazione sono trasparenti (MVP end-to-end, coverage), e questo tutela entrambe le parti da richieste fuori dal dominio del capitolato a metà progetto.

3.7.6 Criticità e rischi. Dopo un paio di iterazioni di analisi e riunioni abbiamo notato delle incoerenze interne, che andranno chiarite al design thinking quando ci verrà assegnato. L'esempio del capitolato («Che tempo fa a Milano oggi?») dipende dal momento, quindi servirlo dalla cache è un errore e non un risparmio: serve un gate di cacheabilità che stabilisca cosa conviene servire e cosa rigenerare. La premessa «frasi vicine = stesso significato» non regge con la similarità del coseno, perché la vicinanza misura il topic e non l'equivalenza (ne abbiamo discusso anche con la proponente), per cui invece di una soglia configurabile il problema andrebbe riformulato in termini di error rate accettabile. Ci sono poi scenari in cui la cache semplicemente non serve: le interazioni da 50–100k token sono sessioni lunghe, spesso con documenti allegati e dipendenti dal contesto, quindi quasi mai cacheabili, mentre gli hit si concentrano su query

brevi single-shot che costano poco; l'80% di ridondanza citato non ha fonti verificabili e il rischio è che la dashboard stessa misuri un risparmio trascurabile. Sono possibili anche leak tra tenant: un hit semantico potrebbe servire una risposta costruita su dati Gold a un utente che non ne ha accesso, e la multi-tenancy compare solo nel FinOps, per cui la cache va partizionata per tenant, livello di sensibilità e permessi. Resta poi la sfida tecnica della concorrenza: più richieste equivalenti in arrivo insieme rischiano di generare tutte un miss e una chiamata al modello, o di scrivere entry in conflitto. Infine LiteLLM si muove in fretta (elenca già un AI Gateway in Rust ancora in beta) e copre in parte i «buchi» con una cache semantica di base e il budget tracking: vanno verificati gli hook Python per tutto il ciclo del progetto e scritto bene il delta rispetto a ciò che esiste.

Sintesi: È risultato il progetto più completo, stimolante e meglio organizzato. Soddisfa a pieno le aspettative di crescita del team.

3.8 C7: SmartDashboard

Descrizione: Il progetto SmartDashboard, proposto da Ergon Informatica, mira a sviluppare una piattaforma di Business Intelligence (BI) self-service potenziata dall'Intelligenza Artificiale. Il sistema estrae i dati grezzi da un gestionale ERP, li consolida in un Data Warehouse analitico e offre una web app per la creazione di dashboard interattive tramite un'interfaccia drag & drop. Il fulcro dell'applicativo è l'integrazione di un layer AI basato su LLM che funge da assistente conversazionale: permette di generare visualizzazioni tramite richieste in linguaggio naturale, estrae insight automatici (anomalie, tendenze) e fornisce spiegazioni contestuali sugli indicatori.
Tecnologie: DuckDB, PostgreSQL/MySQL, Python (Polars/Pandas per l'ETL), React/Angular/Vue, librerie di charting, LLM via API (OpenAI GPT / Anthropic Claude), API REST.

- **Pro:** Architettura modulare a tre livelli ben definita, esplorazione di tecnologie moderne per l'analisi dati (come DuckDB), e un rassicurante approccio di sviluppo incrementale (prima BI, poi AI) delineato dall'azienda.
- **Contro:** Il notevole sforzo richiesto per il Data Engineering (pipeline ETL) potrebbe togliere spazio allo sviluppo dell'app; inoltre, l'uso di LLM commerciali via API limita il lavoro di implementazione IA alla sola integrazione e al prompt engineering.

Sintesi: Si riduce a reinventare la ruota creando l'ennesimo strumento di BI ridondante, senza affrontare vere sfide di sistema o di logica.

3.9 C8: Sonar: Documentazione vivente di uno store Shopify

Descrizione: Il progetto mira a realizzare un'applicazione web che si collega a uno store Shopify enterprise, ne interroga API e codice del tema e ricostruisce in automatico la mappa di cosa è installato, di cosa dipende da cosa e di cosa non serve più. I dati acquisiti tramite scansioni durevoli vengono normalizzati in un modello a grafo, salvati come snapshot immutabili e analizzati da un motore di regole estensibile che segnala orfani e codice morto, indicando fonte e livello di confidenza. Il risultato è una documentazione sempre aggiornata, consultabile via interfaccia web e con diff tra snapshot, esportabile in Markdown/HTML.

Tecnologie: Next.js, Vercel, Supabase, TypeScript, Shopify Admin GraphQL API, Git/GitHub, Drizzle ORM/Prisma, Tailwind CSS, shadcn/ui, React Flow/Cytoscape.js, OpenTelemetry, Vitest, Playwright, MSW, GitHub Actions

- **Pro:** Contesto e-commerce reale e vendibile; output altamente tangibile; stack front-end moderno.
- **Contro:** Vincoli e limiti delle API Shopify; lock-in su piattaforme proprietarie; scarsa flessibilità lato database.

Sintesi: Lavorare dentro ecosistemi proprietari chiusi come Shopify impedisce qualsiasi controllo manuale serio sulle fondamenta relazionali e sull'architettura.

3.10 C9: SpecCheck: Skills OpenCode per tracciabilità tra casi d'uso, requisiti e codice sorgente

Descrizione: Il progetto, proposto da Zucchetti, mira a sviluppare SpecCheck, un sistema di estensioni (skill, tool e plugin) per l'agente di harness OpenCode. Per risolvere il problema della verifica nel "vibe coding", la piattaforma automatizza la tracciabilità end-to-end nel ciclo di vita del software: confronta i casi d'uso con le specifiche dei requisiti e verifica che il codice sorgente implementi fedelmente quanto richiesto, segnalando eventuali disallineamenti tramite una dashboard dedicata.

Tecnologie: OpenCode, LLM commerciali via API o open-source, formati di modellazione e documentazione (Mermaid, XMI, Markdown, PDF), Bash/PowerShell, framework per dashboard testuali (TUI) o grafiche (HTML), framework per test automatici.

- **Pro:** Focus su paradigmi IA di ultima generazione ("Vibe Coding"); automazione avanzata del controllo qualità; creazione di tool (Node/JavaScript) e dashboard visive.
- **Contro:** Allontanamento radicale dalla scrittura di codice esplicito tradizionale; analisi delegata a modelli LLM intrinsecamente non deterministici; lock-in sull'agente proprietario OpenCode.

Sintesi: Sostituisce l'ingegneria del software classica con la programmazione in linguaggio naturale ("Vibe Coding"), spingendo verso un'astrazione totale gestita dall'IA che azzera il tuo controllo manuale e architetturale sul codice.

4 Capitolato Scelto: Caching Semantico e FinOps (C6)

4.1 Motivazione Scelta

4.1.1 Allineamento con le competenze del gruppo. : Le competenze necessarie per requisito sono raggruppate per aree, delle quali Python è il nucleo perché richiede l'usufruimento di LiteLLM come proxy di base, FastAPI o Flask sono consigliati per esporre API ed aderire ad una architettura a microservizi; ovviamente sarà presente un livello di persistenza che potrà essere realizzato via DB relazionali o non, che dunque richiede conoscenze nel modellare realtà via diagrammi e traduzione in tabelle o grafi (anche se l'embedding in questo caso ha struttura esplicita e pgvector gestisce DB vettoriali); per la similarità tra embedding ci saranno richiami ad algebra lineare di base (prodotto scalare e norma). I vincoli di modularità ed estensibilità nel design riprendono i principi di Programmazione ad Oggetti (vari pattern per spingersi oltre il semplice polimorfismo). Le lacune dichiarate invece per maggior parte del gruppo sono: LiteLLM come libreria, vectore store, per alcuni il ML applicato (anche se la teoria è stata accennata in Introduzione del Machine Learning) per cross/bi-encoder ed il frontend per cui alcuni riportano lacune in framework di JavaScript come React e Chart.js, di conseguenza, Streamlit potrebbe essere la soluzione più diretta nel realizzare la dashboard.

4.1.2 Interesse Tecnico e Professionale. La gestione dei costi per gli LLM (o FinOps per l'IA) sta diventando un'esigenza reale, in questo caso la fortuna sta nel fatto che LiteLLM è già in produzione, per cui i requisiti nascono da un contesto enterprise concreto; quello della cache semantica è ancora un problema aperto, da cui un soglia sul coseno non garantisce tasso d'errore fisso, e qui c'è spazio per logica originale che il capitolato chiede oltre la semplice integrazione di componenti. Le competenze apprese valgono più della specificità in cui si applicano, gateway LLM, vector DB, valutazione statistica e applicabilità di modelli computazionali nella macro-soluzione sono fortemente trasversali. Infine il processo di sviluppo richiesto è altamente professionale, sessioni di design thinking, lo User Story Mapping e standup periodici col referente (che nel nostro caso sono coincidenti con la stima del fine sprint, giusto in tempo per fare retrospettiva).

4.1.3 Rapporto Complessità/Fattibilità. La fattibilità è quantomeno alta, infatti il progetto è tutto software con uno stack consolidato e containerizzabile, soprattutto non dipende da hardware e/o dati proprietari, LiteLLM stesso essendo open-source e ben documentato ha punti di estensione personalizzati (per il callback, metodi su cache object). Il rischio tecnico invece è addensato ed è possibile affrontarlo presto. L'idea sta tutta nella capacità di distinguere domande davvero equivalenti, e la PoC che il capitolato impone prima dell'implementazione viene affrontato già entro l'RTB. La separabilità del lavoro è un'altro grande punto, il lavoro può essere diviso bene tra 7 persone per cui sono coperti lavori su: cache, servizio FinOps (via dashboard) ed infrastruttura e qualità del codice. Rispetto alla nostra seconda scelta, il capitolato C3 (progetto FossDot), non vi sono vincoli sulla licenza dei modelli, non è necessario neanche interagire con hardware embedded o SBC, oppure ancora DSP audio, inoltre non è necessario realizzare firware ed i relativi aggiornamenti OTA.

4.1.4 Qualità del rapporto con la proponente. La proponente ha mostrato disponibilità e professionalità nel mantenere un dialogo aperto, quantomeno alcuni dubbi sono stati risolti attraverso lo scambio di mail (del 22/10), successivamente la necessità di una sessione dedicata per domande e riflessioni è stata adempiuta appropriatamente via riunione su Microsoft Teams (del 26/10), inoltre si sono mostrati particolarmente tempestivi nel ritorno della richiesta di firma del nostro primo verbale esterno (del 27/10). Dal capitolato emerge che VarGroup mantiene un supporto stutturato, tant'è che promette sessioni di design thinking in presenza (3h) e successivi follow-up sui requisiti (1h), tre formazioni con video e hands-on di LiteLLM (2h), Embedding e Caching (2h) ed infine FinOps + Dashboard(1,5h). Gli altri due punti fondamentali sono: gli stand-up di allineamento (30m) ed i Q&A ogni due settimane, coincidente alla durata dei nostri sprint, dunque è possibile fare restrospettiva con la proponente.

4.2 Strategie di mitigazione dei rischi

I rischi sono dichiarati e descritti nella sezione sottosottosezione 3.7.6 sono richiamati qui per indicare una strategia di mitigazione, oltrechè accompagnati da un indicatore come bussola per la mitigazione.

Tabella 1: Analisi rischi e mitigazioni

Rischio	**Strategia**	**Indicatore**
Falsi hit (soglia unica sul coseno, domande che dipendono dal momento)	Ricerca dei candidati + verifica con cross-encoder; soglia calibrata su coppie etichettate; filtro che esclude le domande non cacheabili; TTL per classe di domanda	Tasso di falsi hit sul validation set sopra il target concordato
Nessun dato reale per valutare	300–500 coppie etichettate, incluse coppie difficili/ambigue (es. "Windows" vs "Linux"); richiesta a Var Group di log aggregati e anonimizzati (oppure costruzione di un dataset come concordato)	Dataset non pronto entro il secondo sprint
Fuga di dati tra tenant, PII in cache, embedding di dati riservati inviati a OpenAI	Cache partizionata per tenant, livello di sensibilità, modello e system prompt; ADR sulla posizione rispetto all'Anonymizer; embedding locali per il Layer 2; test di isolamento	Fallisce un test di isolamento
Risparmio misurato deludente (la maggior parte dei token sta nei contesti lunghi, poco cacheabili)	Concordare come si calcola il risparmio netto (embedding, verifica, falsi hit); stimare presto l'hit rate atteso	Hit rate stimato sotto la soglia di interesse
LiteLLM cambia rapidamente	Versione fissata; solo punti di estensione documentati; test di integrazione su quella versione; nessun fork	Breaking change alla release successiva

4.3 Piano di formazione preliminare

Per colmare le lacune presentate in sottosottosezione 4.1.1, avremo necessità di percorsi di formazione (non necessariamente comuni) prima che diventino blocchi di tempi morti per il gruppo intero. Per cui sicuramente LiteLLM dovrà essere studiato da tutti (via la documentazione, avvio stack locale per far passare chiamate, ed avere feedback via log per i token), sarà necessario anche un ripasso di Docker Compose e pytest, ed il flusso Git del gruppo (già concordato ed in sperimentazione via Trunk Based Development personalizzato). Per le persone a cui è di interesse la cache semantica verranno richiamati concetti di precision/recall, ma anche di calibrazione, ed apprendimento della meccanica dei sentence-trasformer e cross-encoder (per trarre vantaggio tangibile nell'adottamento di determinate tecnologie), qui sarà richiesto di seguire testi (per ciò che serve), LLM, colleghi maturi nel settore, ed apprendimento attivo con formazione sull'embedding e caching. Anche qui è possibile verificare su un Notebook Jupiter per esempio il confronto del coseno e cross-encoder su 100 coppie di embeddings. Coloro che saranno assegnati al FinOps e dashboard dovranno apprendere gli eventi di spesa di LiteLLM, aggregazioni su

PostgreSQL, documentazione di FastAPI, Streamlit o React+Chart.js; come criterio di verifica è possibile realizzare un prototipo di dashboard modulare su eventi stub o sintetici. Infine per una minoranza sarà necessaria la preparazione sul design thinking e User Story Mapping, oltrechè pipeline CI con soglia di coperatura, ma questo è coperto nel materiale del corso a cui faremo riferimento in caso di dubbi.

4.4 Impegno orario stimato

Il motivo per cui i ruoli pesano così è che inizialmente per gli Analisti ci sarà grosso lavoro nel lavoro di un design thinking coerente, oltrechè User Story Mapping e requisiti precisi da fissare (per esempio error rate acceptance vs soglia della similarità tarabile, metriche esatte di risparmio e priorità tra i due prodotti); dopodichè Progettisti dovranno tracciare lo storico delle scelte tecnologiche ed adottamento di schemi via un ADR (per esempio la decisione architetturale del partizionamento multi-tenant, il riuso stesso di LiteLLM) e modellazione dei dati FinOps. Per i Verificatori ci sarà il compito di mantenere coerente il vincolo del 70% di copertura, ma anche della separazione di classi di test deterministici da stime probabilistiche. I Programmatori si dovranno occupare dello sviluppo dei due servizi, dashboard ed integrazione end-to-end, mentre Amministratori si occuperanno dello stack Docker (con particolare attenzione alla config drifting) e la impostazione della pipeline CI. Infine i Responsabili agiranno da collante tra le parti e dunque coordinamento con i vari referenti, parteciperanno diretteamente ai standup e sessioni di Q&A.

Da notare la distrubuzione del tempo, in 7 il carico prima dell'RTB(11 gennaio 2027) è circa dal 35-40% delle ore totali, considerazioni sulla presenza di esami tra gennaio e febbraio ed eventuali imprevisti sono per indicare con alta probabilità ci sarà un calo sulla capacità di forza lavoro, ed in tutto questo, la stima delle ore di formazione è esclusa.

5 Conclusioni e Scelta Finale

Al termine del processo di analisi comparativa, il gruppo **Bit Crusher** ha deliberato di presentare formale candidatura per l'assegnazione del seguente progetto didattico:

C6: Sistema AI di caching semantico e dashboard FinOps

Proposto da **Var Group S.p.A.**

Motivazioni della Scelta:
La decisione è maturata a seguito di una valutazione approfondita su più livelli. Il capitolato offre un perfetto bilanciamento tra richiesta di competenze tecniche (AI Embedding, Vector Search) ed ingegneria del software applicata (estensione di LiteLLM, architetture a microservizi, ottimizzazione FinOps). Le sfide poste dal progetto si allineano eccellentemente con l'interesse del gruppo, tant'è che gran parte dei membri mostrano curiosità e coinvolgimento attivo nelle discussioni. Infine, l'approccio metodologico proposto da Var Group (Design Thinking, ADR, focus sul testing automatico) garantisce un ambiente di lavoro di alta qualità formativa.

Il Responsabile di Progetto

